

Proyecto Final

Tema: Embeddings de palabras con optimización restringida usando un fragmento de *Don Quijote de la Mancha*

Materia: Optimización numérica

Integrantes: Adrian Janus Gonzalez Adamuz

1. Introducción

En este proyecto trabajamos un problema sencillo de procesamiento de lenguaje natural visto desde el enfoque de optimización. La idea principal fue construir representaciones vectoriales de palabras, también llamadas *embeddings*, a partir de un fragmento de *Don Quijote de la Mancha*. En lugar de usar un corpus grande o un modelo complejo, tomamos un conjunto pequeño de palabras clave del texto y entrenamos sus vectores con una función de pérdida tipo *skip-gram* con muestreo negativo.

La parte importante del proyecto no fue el tamaño del modelo, sino la forma en que se impusieron restricciones durante el entrenamiento. Se compararon dos estrategias: penalización cuadrática y barrera logarítmica. Las dos buscan resolver el mismo problema, pero se comportan distinto cuando intentan mantener la solución dentro de la región factible. Por eso el proyecto encaja bien con el tema de optimización restringida.

Se decidió dejar solo el caso de *Don Quijote* porque era el ejemplo más claro para explicar resultados. El experimento con notas musicales servía como comparación sintética, pero para la entrega final hacía más ruido que ayuda. Con una sola fuente de datos, el código queda más limpio, la exposición es más fácil de seguir y el reporte se concentra en una idea concreta.

2. Objetivo

El objetivo general fue entrenar embeddings de palabras a partir de un fragmento literario y comparar dos métodos de optimización restringida sobre el mismo problema.

Los objetivos particulares fueron los siguientes:

1. Construir un vocabulario pequeño con personajes y conceptos relevantes del fragmento.
2. Generar pares de co-ocurrencia usando una ventana de contexto.
3. Definir una función de pérdida que acerque palabras que aparecen en contextos similares.
4. Imponer restricciones geométricas sobre los embeddings.
5. Comparar el comportamiento de la penalización cuadrática y la barrera logarítmica.
6. Interpretar si las relaciones aprendidas por el modelo tienen sentido dentro del texto.

3. Descripción del problema

Se trabajó con un fragmento breve de *Don Quijote de la Mancha* donde aparecen palabras centrales como **quijote**, **sancho**, **rocinante**, **dulcinea**, **molinos**, **viento**, **gigantes** y **escudero**. Después de limpiar el texto y quitar palabras vacías, se conservó un vocabulario de catorce términos:

quijote, sancho, rocinante, dulcinea, molinos, viento, caballero, aventuras, mancha, gigantes, batalla, escudero, novela y juicio.

Con estas palabras se armó una secuencia filtrada y luego se generaron pares positivos con una ventana simétrica de tamaño 2. Eso significa que, para cada palabra del vocabulario que aparece en el fragmento, se tomaron como contexto las palabras cercanas. Si dos palabras aparecen varias veces próximas una de otra, el peso de ese par aumenta.

La intuición del modelo es simple: si dos términos comparten contexto, sus vectores deberían quedar cerca en el espacio. Así, por ejemplo, era razonable esperar que **quijote**, **rocinante** y **dulcinea** terminaran relativamente próximos, o que **sancho** y **escudero** mostraran afinidad.

4. Modelo matemático

Cada palabra del vocabulario se representa con un vector en dimensión 2. Se eligió dimensión 2 porque hace más fácil graficar e interpretar los resultados. Si llamamos E a la matriz de embeddings, cada fila E_i corresponde a una palabra.

Sea $V = \{1, \dots, n\}$ el vocabulario y sea $E \in \mathbb{R}^{n \times d}$ la matriz de embeddings, con $d = 2$ en nuestro experimento. Si (i, j) es un par positivo de centro y contexto, el modelo busca que el producto interno $E_i^\top E_j$ sea alto. Si (i, k) es un par negativo generado por muestreo aleatorio, el modelo busca que ese producto interno sea bajo.

La función de ajuste a datos sigue la lógica de *skip-gram* con muestreo negativo y puede escribirse como:

$$\mathcal{L}_{data}(E) = - \sum_{(i,j) \in \mathcal{D}_+} w_{ij} \log \sigma(E_i^\top E_j) - \sum_{(i,k) \in \mathcal{D}_-} \log \sigma(-E_i^\top E_k),$$

donde σ es la función sigmoide, $\mathcal{D}_+$ es el conjunto de pares positivos, $\mathcal{D}_-$ el conjunto de pares negativos y w_{ij} es el peso asociado a la frecuencia del par positivo. En términos simples, el modelo trata de dar puntajes altos a los pares de palabras que sí aparecen como contexto y puntajes bajos a pares aleatorios.

Además de ajustar los datos, se impusieron dos restricciones:

1. La norma cuadrada de cada embedding debía ser menor o igual que 1.
2. Las dos dimensiones del espacio debían mantenerse lo más desacopladas posible mediante una condición de ortogonalidad global.

De forma compacta, el problema de optimización queda así:

$$\min_E \mathcal{L}_{data}(E)$$

sujeto a

$$\|E_i\|_2^2 \leq c \quad \forall i, \quad E^\top E = I_d,$$

con $c = 1$ e I_d la matriz identidad de dimensión d .

Estas restricciones sirven para evitar soluciones arbitrarias y para mantener una geometría más estable. La primera impide que algunos vectores crezcan demasiado. La segunda evita que ambas dimensiones terminen representando casi lo mismo.

Si se escribe el problema desde el enfoque clásico de optimización restringida, el Lagrangiano es:

$$\mathcal{L}(E, \lambda, \Lambda) = \mathcal{L}_{data}(E) + \sum_{i=1}^n \lambda_i (\|E_i\|_2^2 - c) + \langle \Lambda, E^\top E - I_d \rangle,$$

donde $\lambda_i \geq 0$ son multiplicadores asociados a las restricciones de norma y Λ es una matriz simétrica asociada a la restricción de ortogonalidad. Esta formulación es importante porque conecta directamente el experimento con el marco teórico del curso: no solo se quiere ajustar datos, sino hacerlo respetando restricciones geométricas bien definidas.

5. Métodos de solución

Se compararon dos métodos.

5.1 Penalización cuadrática

En este enfoque, las restricciones no se obligan de manera exacta en cada iteración, sino que se agrega un costo cuando se violan. Si un vector rebasa la norma permitida o si la matriz no cumple bien la ortogonalidad, la función objetivo aumenta y el optimizador trata de corregirlo.

La función usada en este caso fue:

$$\min_E \mathcal{L}_{data}(E) + \alpha \|E^\top E - I\|_F^2 + \beta \sum_i \max(0, \|E_i\|^2 - c)^2.$$

Aquí, α controla qué tanto se castiga la falta de ortogonalidad y β qué tanto se castiga la violación de la norma.

La ventaja de esta idea es que es fácil de implementar y suele avanzar rápido. La desventaja es que puede acercarse al borde de la región factible o incluso pasar ligeramente por fuera durante el entrenamiento.

5.2 Barrera logarítmica

En este caso, el método agrega un término que crece mucho cuando una solución se acerca al borde de la restricción de norma. Eso obliga al modelo a mantenerse dentro de la región factible. En la práctica, este enfoque suele ser más conservador.

La función objetivo para este método fue:

$$\min_E \mathcal{L}_{data}(E) + \alpha \|E^\top E - I\|_F^2 - \mu \sum_i \log(c - \|E_i\|^2).$$

El término con logaritmo actúa como barrera interior: cuando $\|E_i\|^2$ se acerca a c , el castigo crece de manera fuerte. Por eso este método tiende a conservar más holgura.

La ventaja es que mantiene mayor holgura respecto a la restricción. La desventaja es que puede requerir un ajuste más cuidadoso del parámetro de barrera y del paso de aprendizaje.

6. Implementación

El código final se dejó en un solo archivo llamado `quijote_minimo.py`. Ese script hace lo siguiente:

1. Define el fragmento de *Don Quijote*.
2. Tokeniza el texto y filtra palabras vacías.
3. Conserva un vocabulario pequeño y fácil de interpretar.
4. Construye pares positivos con ventana de contexto.
5. Entrena embeddings con penalización cuadrática.
6. Entrena embeddings con barrera logarítmica.
7. Imprime métricas finales.
8. Muestra vecinos cercanos para interpretar relaciones semánticas.
9. Guarda una gráfica llamada `embeddings_quijote.png`.

Esta versión es más corta que el notebook original y se puede ejecutar directamente con:

```
python3 quijote_minimo.py
```

7. Resultados

Al ejecutar el script se obtuvieron los siguientes resultados:

Método	Data loss	Error de ortogonalidad	Máxima norma cuadrada	Holgura mínima
Penalización	1.3454	0.0138	0.2887	0.7113
Barrera	1.3473	0.0124	0.2288	0.7712

Lo primero que se observa es que ambos métodos ajustaron de manera parecida los datos. La pérdida final fue casi la misma, así que en términos de co-ocurrencias no hubo una diferencia grande.

La diferencia más clara apareció en la parte de factibilidad. El método de barrera terminó con menor norma máxima y con mayor holgura mínima. Eso significa que dejó los embeddings más alejados del borde permitido. También obtuvo un error de ortogonalidad ligeramente menor. En este experimento, la barrera se comportó como un método más ordenado respecto a las restricciones.

La penalización cuadrática, por su parte, también produjo una solución válida, pero más cercana al límite de norma. No fue una mala solución; simplemente fue menos conservadora.

8. Interpretación de vecinos cercanos

Para revisar si el embedding realmente captó relaciones del texto, se analizaron los vecinos más cercanos obtenidos con el método de barrera. Algunos resultados fueron:

- quijote -> dulcinea, molinos, rocinante
- sancho -> escudero, viento, molinos
- rocinante -> dulcinea, quijote, molinos
- molinos -> quijote, dulcinea, viento
- gigantes -> batalla, sancho, escudero

Estos resultados no son perfectos, pero sí tienen sentido. Quijote aparece cercano a Rocinante y Dulcinea, que son dos referencias centrales del personaje. Sancho aparece ligado a escudero, lo cual es correcto dentro del relato. Molinos y viento también terminan relacionados, como era de esperarse por el episodio seleccionado.

Algo importante es que el vocabulario es muy pequeño y el texto también. Por lo tanto, no tiene sentido exigir una estructura semántica tan fina como la de un modelo entrenado con miles de documentos. Aun así, el experimento sí muestra que con un problema pequeño ya se puede observar el efecto de las restricciones sobre la geometría de la solución.

9. Discusión

Desde el punto de vista de optimización, el proyecto sirve para ver una idea concreta: dos métodos distintos pueden ajustar casi igual los datos y aun así diferir en la forma en que respetan las restricciones. Ese punto es justo el centro de la comparación.

Si solo miráramos la pérdida, podríamos concluir que no hay diferencia importante entre penalización y barrera. Pero al revisar la norma máxima, la holgura y la ortogonalidad, sí aparece una diferencia técnica. La barrera deja una solución más interior. La penalización deja una solución competitiva, aunque más cercana al borde.

También conviene mencionar las limitaciones del trabajo. Se utilizó un fragmento corto, no la novela completa. El vocabulario se eligió manualmente para que la visualización fuera clara. La dimensión se fijó en 2 para poder graficar, aunque una dimensión mayor podría capturar mejor algunas relaciones. Además, el muestreo negativo introduce algo de variación, por lo que los números pueden cambiar ligeramente en otras corridas si se cambia la semilla o la configuración.

A pesar de eso, el proyecto cumple bien como ejercicio final porque conecta tres cosas al mismo tiempo: texto, representación vectorial y optimización restringida. El experimento es pequeño, pero suficiente para discutir resultados con fundamento.

10. Conclusiones

El proyecto mostró que es posible construir embeddings interpretables a partir de un fragmento de *Don Quijote de la Mancha* usando un modelo simple y dos enfoques de optimización restringida.

Los dos métodos dieron un ajuste parecido en la función de pérdida, pero la barrera logarítmica respetó mejor la región factible. En números, logró menor error de ortogonalidad, menor norma máxima y mayor holgura mínima. Por eso, para este problema en particular, se puede decir que la barrera produjo una solución más limpia desde el punto de vista de restricciones.

También se observó que las relaciones semánticas básicas del texto sí aparecen en los vecinos cercanos del embedding. No se trata de un modelo grande ni de un sistema de lenguaje avanzado, pero sí de una prueba clara de que la información de contexto puede organizar palabras de forma coherente.

En resumen, el trabajo cumplió el objetivo planteado: formular un problema sencillo de embeddings, resolverlo con dos métodos vistos en optimización y comparar sus resultados con base en ajuste y factibilidad.

11. Trabajo futuro

Si se quisiera extender el proyecto, las mejoras más directas serían:

1. Usar más capítulos de la novela para tener estadísticas más estables.
2. Probar otras dimensiones para el embedding.
3. Repetir el experimento con varias semillas y reportar promedios.
4. Agregar otra métrica de interpretación además de vecinos cercanos.

12. Referencias

1. Cervantes Saavedra, M. de. *Don Quijote de la Mancha*.
2. Mikolov, T., Chen, K., Corrado, G., y Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space*.
3. Nocedal, J., y Wright, S. J. (2006). *Numerical Optimization*.